

Project Report: Arbitrary Precision Arithmetic in Java

Harsha (CS24BETCH11011)

Given Question

The question was to implement a custom library in Java that supports arithmetic operations on arbitrarily large integers and floating-point numbers. These numbers had to be input as strings. We had to support all four basic operations: addition, subtraction, multiplication, and division, and the precision should be preserved—up to 30 digits after the decimal for float division. Additionally, we were required to provide a main class that could accept command-line arguments and produce the correct result.

Main Idea of the Project

When I first read the problem, I realized the whole point was to understand how arithmetic works internally. Since we couldn't use any shortcut libraries, I had to think about numbers as strings and simulate how we do arithmetic by hand. This meant building everything from scratch:

- I needed a way to store and manipulate numbers that might be thousands of digits long.
- I thought of using strings because they can store large sequences of digits easily.
- Every operation (add, sub, mul, div) would need to be done the same way we do it on paper, like adding digits right to left and carrying over when needed.
- For floats I had to make sure the decimal part was handled separately but also correctly positioned in the final result.

Structure of the Project

The whole program is mainly divided into 4 parts:

- **AInteger.java** – For handling all integer operations using string manipulation.
- **AFloat.java** – For handling decimal numbers with high precision.
- **MyInfArith.java** – A command-line interface that takes input and runs the correct operation.
- **run_project.py** – A simple Python script that helps in compiling and running the program easily.

Detailed Steps and Logic

AInteger.java

This class implements all the arithmetic operations on large integers by treating them as strings.

Addition:

- Start from the last digit of both numbers and move leftward. Sum the digits and keep track of the carry (if any).
- Use the ASCII values of characters to convert the digits to integers, perform the addition, and store the result.
- After summing the digits, reverse the result (since we start from the least significant digit) to get the correct number.
- The cases where the numbers have different lengths is taken care by adding the smaller number with zeros at the beginning.

Subtraction:

- First, check which number is larger this will determine the sign of the result.
- Then, subtract digits starting from the least significant digit, borrowing when necessary (similar to manual subtraction).
- Same as addition, the result is reversed at the end to correct the digit order.
- Handling negative numbers involves checking the sign of the larger number before performing the subtraction and handled according to the case.

Multiplication:

- This operation uses the standard school method of multiplication where we multiply each digit of the first number by each digit of the second number.
- Each partial product is stored in an array, and appropriate shifts are made for each successive digit multiplication.
- After all partial products are calculated, carry over is handled and the final result is formed.
- The result is then reversed to correct the order.

Division:

- The division operation is implemented similarly to long division. The remainder is computed step by step.
- After performing the division and obtaining the quotient, we subtract the product of the divisor and the quotient from the current number.
- The result is continuously updated until the remainder is smaller than the divisor.
- Special care is taken to handle the fractional part for division results, especially ensuring that the division result is precise up to 30 decimal places.

AFloat.java

Handling floating-point numbers was more complex due to the presence of the decimal point. The solution involves splitting the input into integer and fractional parts and manipulating them as integers.

- Split the input into two parts: the integer part and the fractional part (separated by the decimal point).
- If the fractional part is shorter than the integer part, pad it with zeros to make them equal in length.
- After padding, concatenate both parts and treat the entire number as an integer string.
- Perform arithmetic (addition, subtraction, multiplication, division) as if they were integers, using the logic defined in `AInteger.java`.
- For division, add 30 zeros to the fractional part before performing the operation to ensure precision.
- After performing the integer arithmetic, insert the decimal point back at the correct position in the result based on the number of digits in the fractional part.

MyInfArith.java

This is the main entry point of the program, responsible for interacting with the user and processing command-line inputs.

- The program expects four arguments: the type of number (either `int` or `float`), the operation to perform (add, sub, mul, div), and the two numbers involved.
- Based on the type argument, the program decides whether to initialize an `AInteger` or `AFloat` object.
- The corresponding arithmetic method (add, sub, mul, or div) is called on the objects.
- The result is printed in the required format, ensuring the correct number of decimal places for floating-point operations.
- Error handling is built into the program to deal with invalid inputs, including incorrect number types, operations, and malformed input values.

Python Script (run_project.py)

This script provides an easy way to compile and run the Java program, streamlining the development and testing process.

- The script first compiles all the Java files into a build directory. This is done using the `javac` command.

- It prompts the user for the necessary command-line arguments. For example, it might ask the user to input something like `float mul 5.5 2.3`.
- The script then runs the compiled Java program using the `subprocess.run()` method, which allows for seamless execution of the command with the provided arguments.
- The script helps automate testing and ensures that the program runs with different inputs without manual intervention.
- The Python script also allows for easy debugging by automating the running of test cases and capturing output.

Testing

Assignment Test Cases:

- `java MyInfArith int add 23650078224912949497310933240250 429397832624671137983863`
→ 66589861487380063295697317641748
- `java MyInfArith int sub 3116511674006599806495512758577 577452423003463811444464`
→ -54628730626339781337950941125431
- `java MyInfArith float mul 6400251.9377695 2326541.6827934` → 14890452913599.971745
- `java MyInfArith float div 244727.15202 75964.3891` → 3.221603634537752111008551505

Custom Tests:

- `java MyInfArith int mul 123456789 987654321` → 121932631112635269
- `java MyInfArith float div 5.5 2` → 2.750000000000000000000000000000

Positive Aspects

- **Deepened Understanding:** Implementing arithmetic manually helped me understand how basic operations actually work at the algorithmic level, beyond what high-level languages provide.
- **No Library Dependence:** The entire system runs without using built-in classes like `BigInteger` or `BigDecimal`, proving the logic is fully custom and under complete control.
- **High Precision:** Floating-point operations maintain precision up to 30 digits.
- **Modular Design:** The use of separate classes (`AInteger`, `AFloat`, and `MyInfArith`) made the project organized and easy to handle.
- **Cross-Language Practice:** Combining Java for logic and Python for automation peaked my interest

limitations

- **Slower Performance:** Since all operations are string-based and done manually, performance is significantly slower compared to built-in numeric types, especially with very large numbers.
- **Limited Error Handling:** While the program handles basic invalid input, it could be more robust in edge cases like division by zero, negative float inputs, or malformed strings.